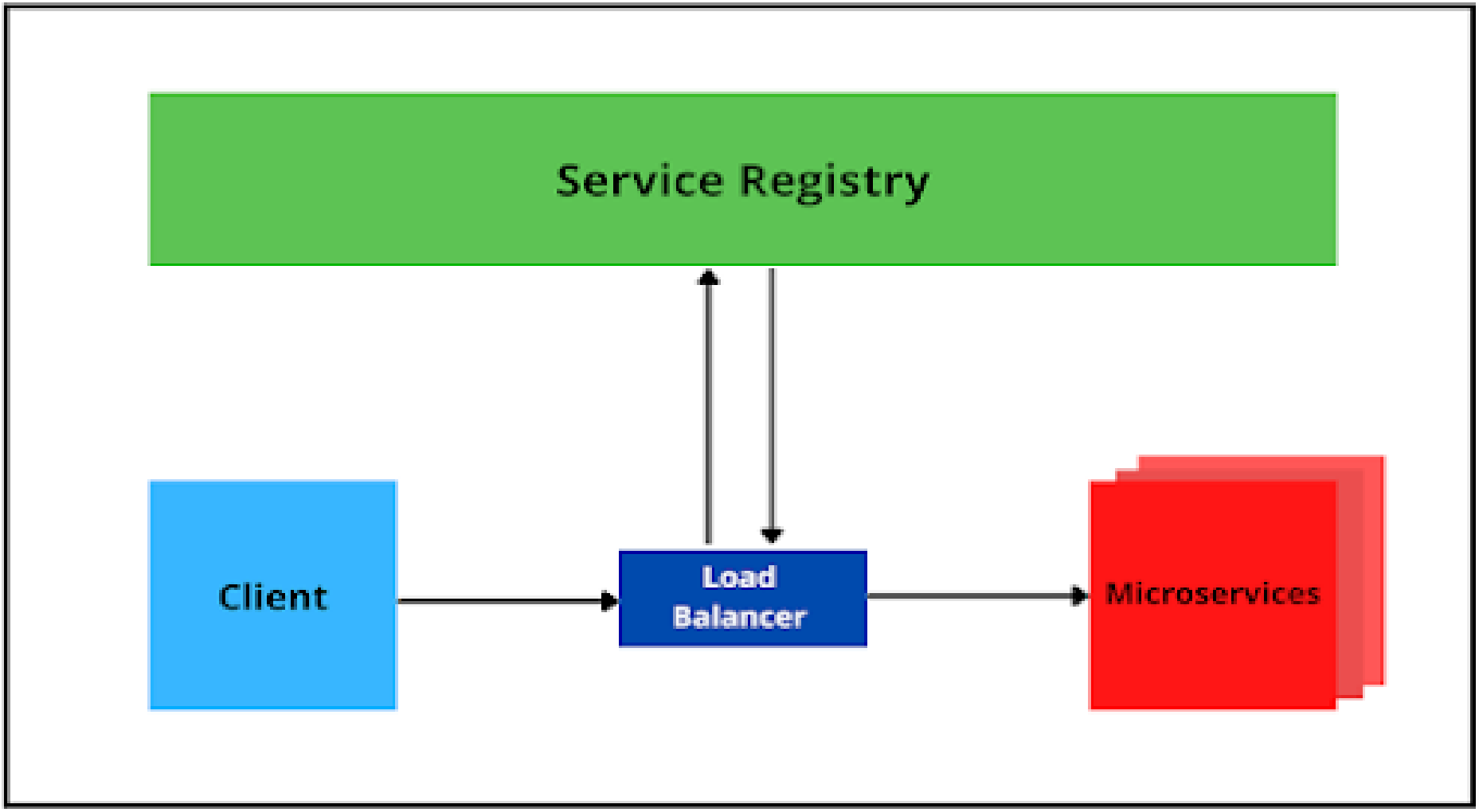
Micro Services

- Day 2

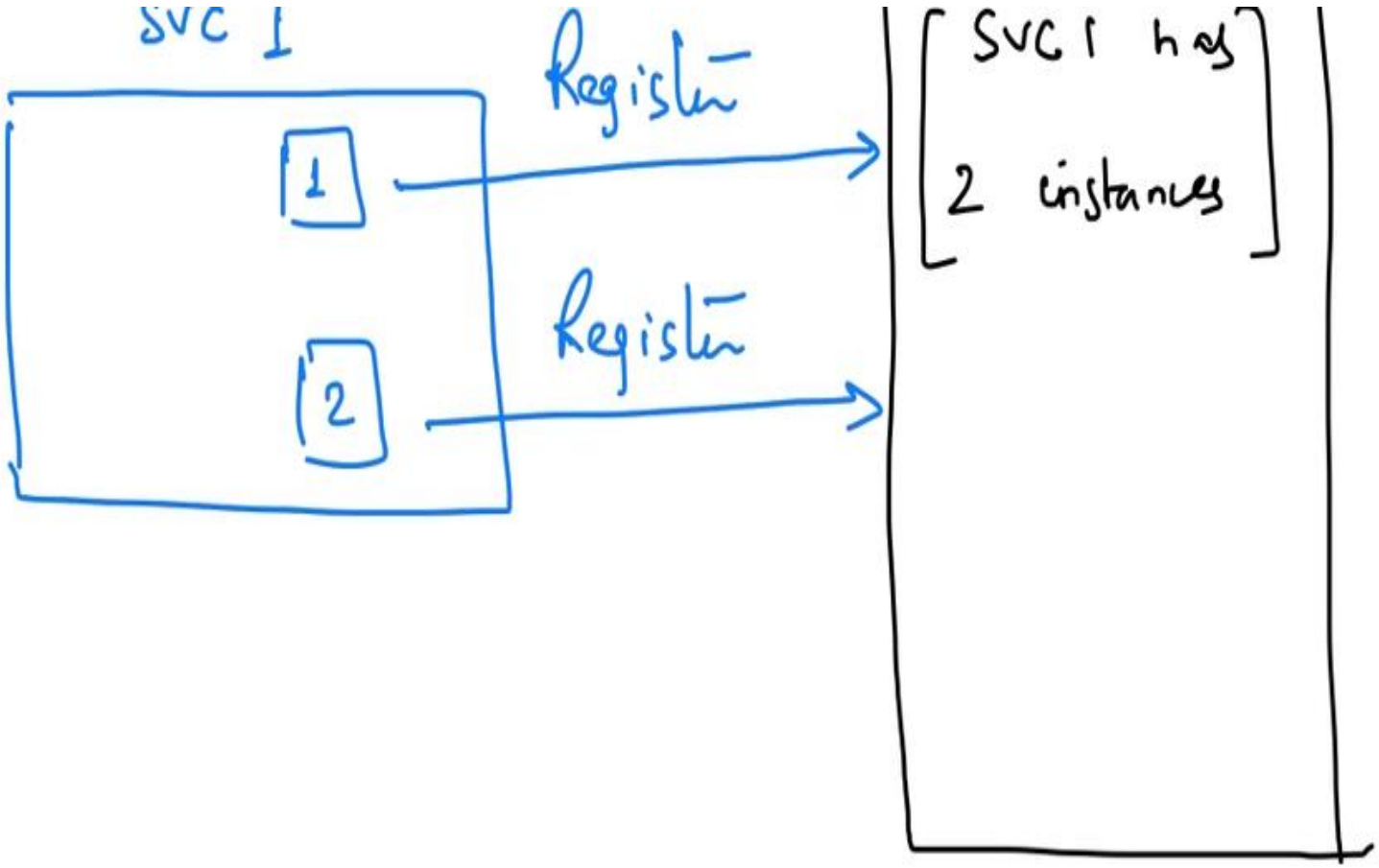
Service Discovery

- **Service Registration**: The process where a service instance notifies the Service Registry of its network location (IP, port) and metadata upon starting up.
- **Service Discovery:** The process where a client (service) queries the Service Registry to find the available network locations of another service it needs to talk to.
- **Service Registry:** The centralized database (e.g., Netflix Eureka, Consul, Nacos) that stores this information.
- **Service discovery** is the **process** of automatically detecting devices and services on a computer network. This reduces the need for manual configuration by users and administrators.
- **Service registration** and **Service discovery** are two closely related parts of the same mechanism, often managed by a single tool like Netflix Eureka. Registration is the action of a service adding its location to a database upon startup, while Discovery is the process of finding that location to make a request.

Diagrammatic Representation



contd..



How They Work Together

- **Start-up (Registration):** When "Service A" starts, it tells the Registration Service, "I am here, my IP is X".
- **Lookup (Discovery):** When "Service B" needs to call "Service A", it asks the Discovery Service, "Where is Service A?".
- **Communication:** The Discovery Service provides the location, and "Service B" calls "Service A" directly.
- **Why Separate Them?**
  - ✓ **Dynamic Environments**: In cloud environments, services scale up, down, or fail constantly. Registration updates the registry in real-time.
  - ✓ **Decoupling:** Services don't need to know the hardcoded IP addresses of other services; they only need to know the location of the Discovery Service.
  - ✓ Common implementations of this pattern include Netflix Eureka, HashiCorp Consul, or Alibaba Nacos.

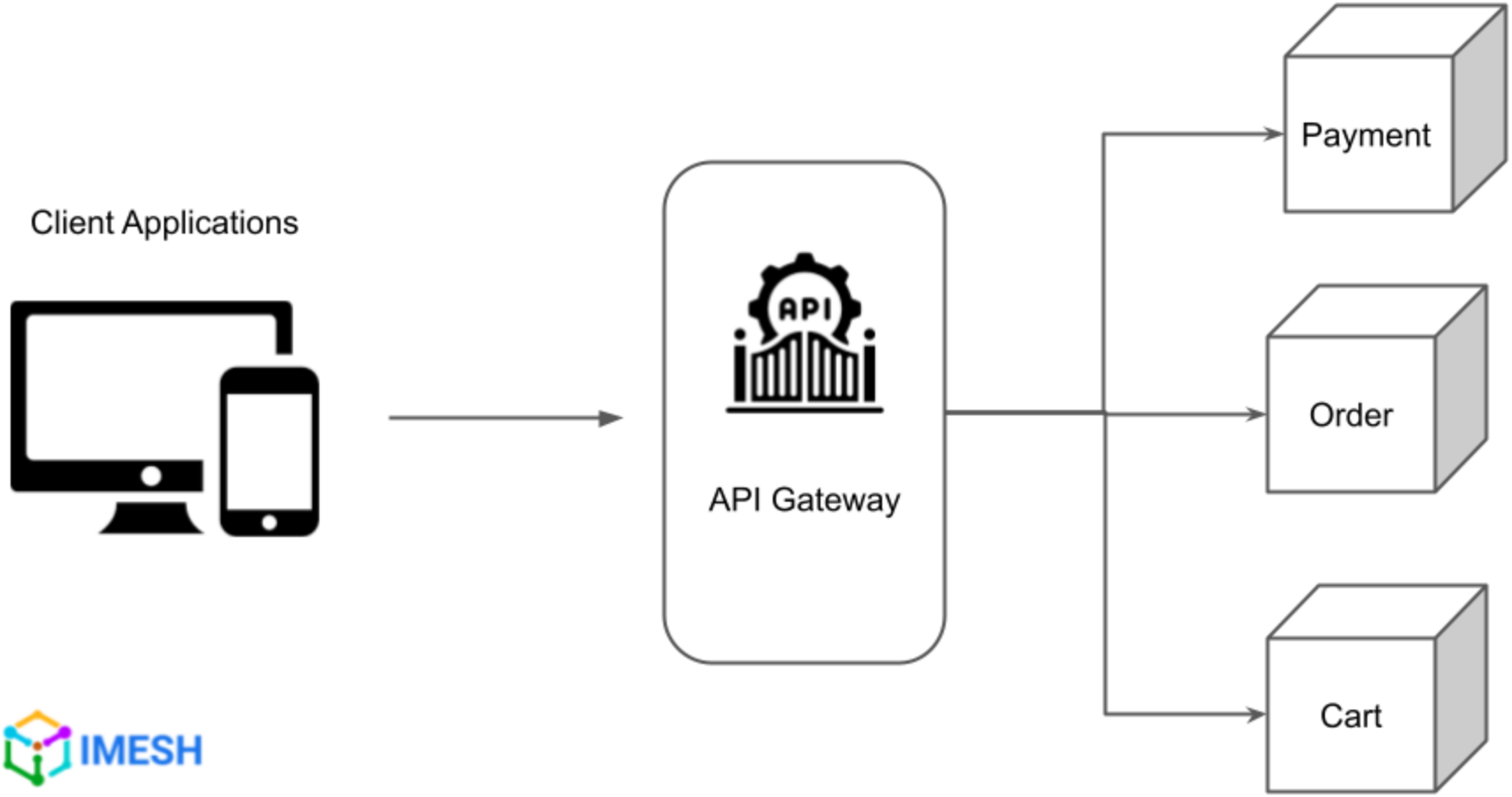
Types of Service Discovery

- There are two types of Service Discovery
  a. Client-Side Service Discovery
  b. Server-Side Service Discovery
- Point to Remember:
  a. Client-Side Service Discovery Example: Netflix Eureka, Zookeeper, Consul
  b. Server-Side Service Discovery Example: NGNIX, AWS ELB

API gateways

- API Gateway is a centralized entry point that manages client requests and directs them to the appropriate backend services. It simplifies communication between clients and multiple microservices while enforcing security and performance policies.
- By providing an API for each client, API gateways encapsulate the complexity of the underlying system and let clients talk to it instead of calling specific services. They also perform security checks (authentication and authorization) before the traffic reaches the service, thus leaving services to focus on their core functionalities.
  - ✓ Handles request routing, authentication, and rate limiting.
  - ✓ Acts as a reverse proxy, hiding internal service complexity.
  - ✓ Provides a unified and consistent API interface for clients.
  - ✓ ***Example:*** *In an e-commerce system, a single API Gateway can route requests for product details, user orders, and payment processing to separate microservices, while also checking user authentication and limiting request rates.*

contd..



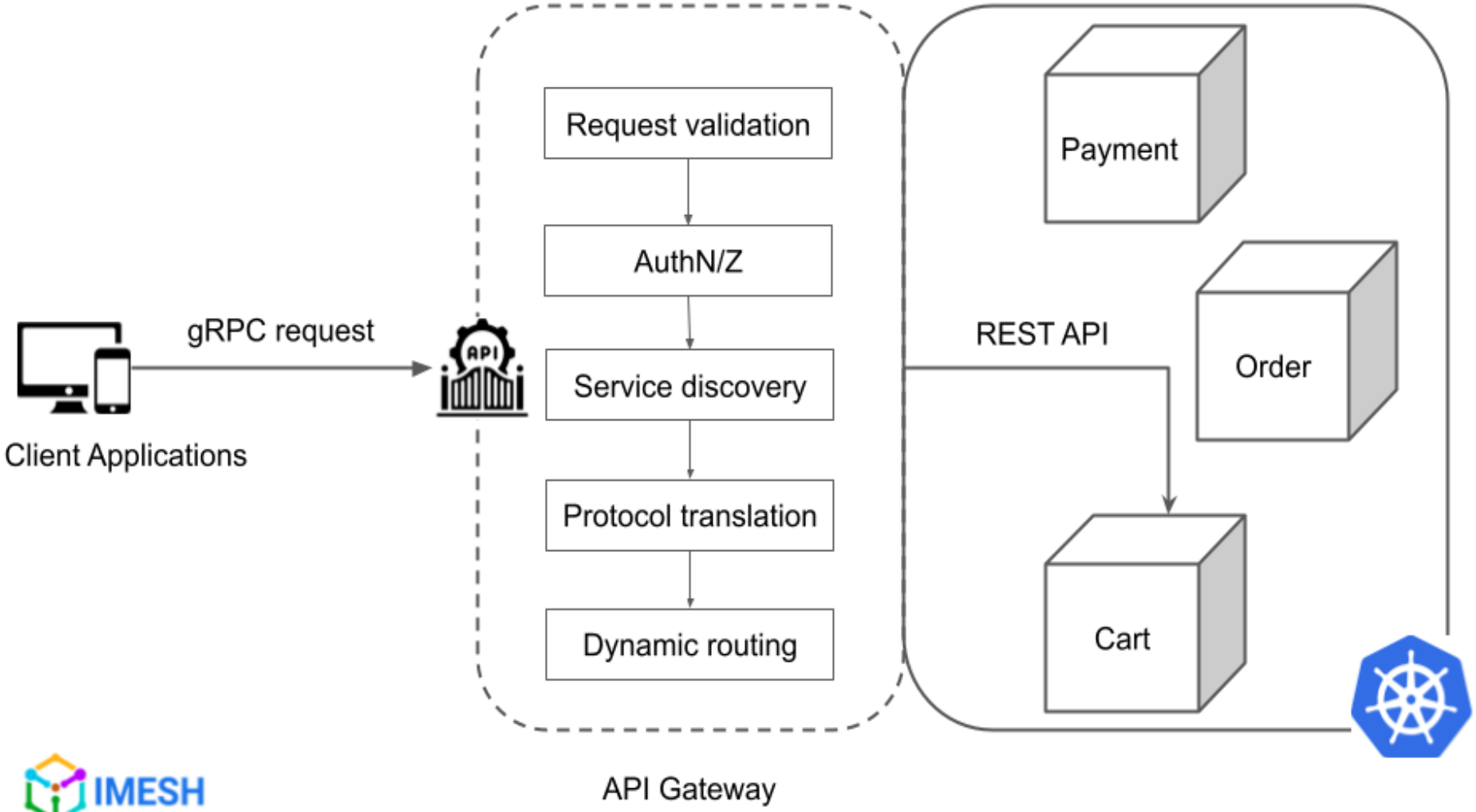
Challenges with direct C2MS communication

- **API gateway traffic flow –** API gateways can read and understand the HTTP messages so can apply filters or take actions on the traffic. Its a L7 proxy that abstracts the traffic management out of front-end microservices, which are usually requested by clients.
- **The problem of service discovery and traffic routing –** the clients have to know the specific endpoints of service instances. Keeping track of endpoints adds complexity for clients because of the dynamic (de)scaling of services. Also, scaling becomes an issue as it will require configuration changes on the client side. Routing traffic based on attributes like geography (geo-routing), is hard to configure when clients invoke services directly.
- **Security concerns –** exposing service endpoints for clients causes security concerns. Increases the attack surface for intruders and leaves backend services prone to threats, e,g. packet sniffing, man-in-the-middle attack, etc. Also it puts the burden of authenticating and authorizing API calls on services, rather than focus on business logic.

contd..

- **Diverse protocols affecting interoperability –** flexibility of microservices architecture lets developers build services using diverse language – Python, Java, Go and implement those services in different API types, such as REST, gRPC, and others. It then becomes a requirement for the client to understand and communicate using different protocols. This adds additional complexity as the client application will require more code and logic.
- **Latency caused by round trips –** Some attributes on client page e.g. product pricing, quantity, and reviews will be deployed as different services in the backend. If the client is invoking services directly, it will have to make separate requests for each service (product price, reviews, quantity, etc.) to retrieve the required information, since there is no mechanism to cache a response from the upstream services. These calls add to the overhead of establishing multiple connections, and the round trips caused by these network requests add to latency and suboptimal user experience.

Diagrammatic Representation



Feign Client

- A **Feign Client** (specifically OpenFeign used with Spring Boot) is a **declarative REST client**. It lets one microservice call another **just by writing an interface**, instead of manually using RestTemplate or WebClient.
- RestTemplate.getForObject("http://PRODUCT-SERVICE/product/1", Product.class);

- @FeignClient(name = "PRODUCT-SERVICE")

```
public interface ProductClient {
    @GetMapping("/product/{id}")
    Product getProductById(@PathVariable int id);
}
```

- Spring automatically handles the HTTP call.

contd..

- No boilerplate HTTP code
- Clean & readable
- Works with Eureka + API Gateway
- Supports load balancing
- Note :

  Feign with name= **requires load balancing**,
  Feign with url= **does not**

contd..

- Notable API Gateways
- Netflix API Gateway: Zuul
- Amazon API Gateway
- Kong API Gateway
- Apigee API Gateway
- MuleSoft
- Potential Drawbacks:

✓ Single Point of Failure: If the gateway goes down, the entire system can fail.

✓ Potential Bottleneck: High-volume traffic can overwhelm the gateway if not scaled properly.